

# **APPLICATION DEVELOPMENT**

**Agricultural Cooperative and Farmer Producer  
Organisation Management System**

## **FRONTEND DOCUMENTATION**

**VIVIDHAN L**

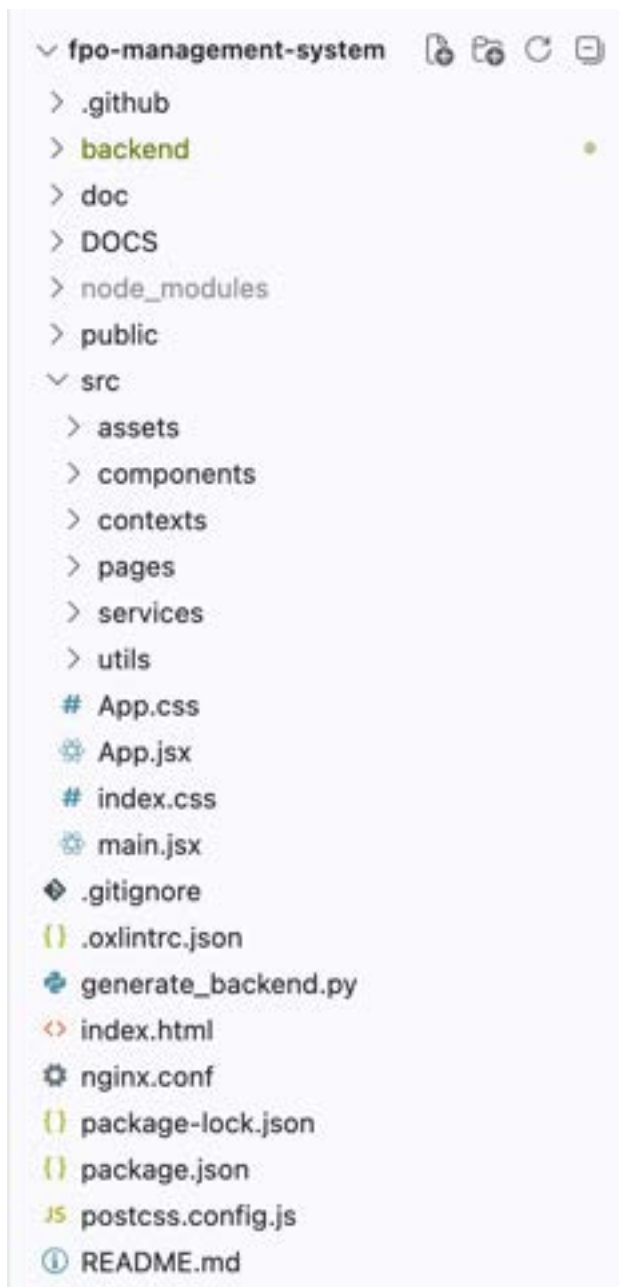
**727824TUIT258**

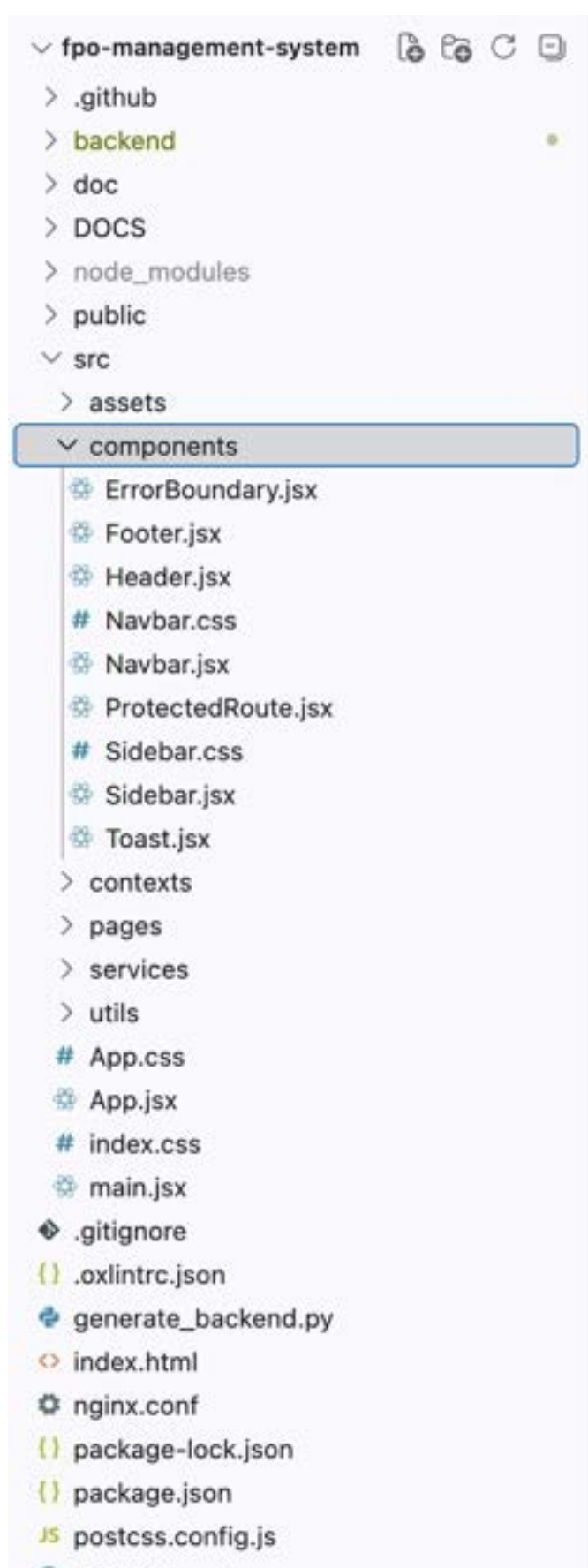
**III IT G**

# Agricultural Cooperative and Farmer Producer Organisation Management System

## FRONTEND:

### 1. Folder Structure:





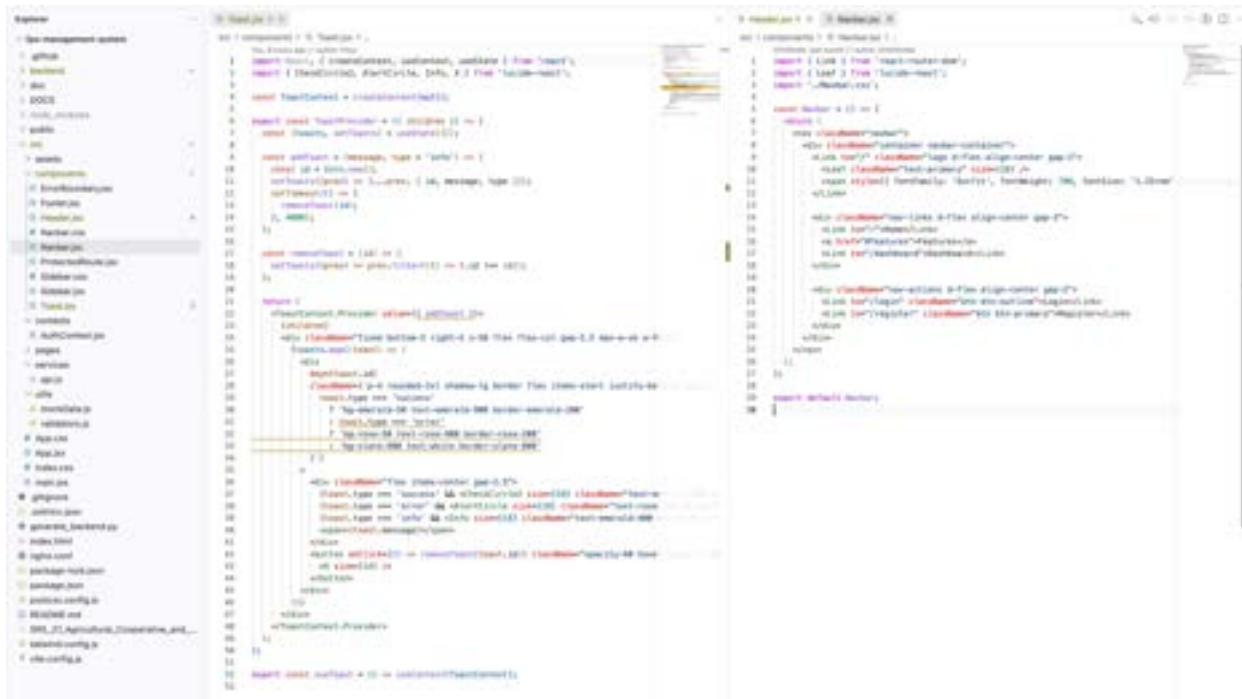
▼ fpo-management-system

- > .github
- > backend
- > doc
- > DOCS
- > node\_modules
- > public
- ▼ src
  - > assets
  - > components
  - > contexts
  - ▼ pages
    - Analytics.jsx
    - # Auth.css
    - Compliance.jsx
    - # Dashboard.css
    - DashboardLayout.jsx
    - DashboardOverview.jsx
    - Finance.jsx
    - # Home.css
    - Home.jsx
    - Login.jsx
    - MemberPortal.jsx
    - Members.jsx
    - Procurement.jsx
    - Register.jsx
    - Sales.jsx
    - Settings.jsx
    - Warehouse.jsx
  - > services
  - > utils
  - # App.css
  - App.jsx
  - # index.css
  - main.jsx
- ◆ .gitignore
- { } .oxlintc.json
- generate\_backend.py
- < index.html
- nginx.conf
- { } package-lock.json
- { } package.json
- postcss.config.js
- ① README.md

## 2. Components (frontend/src/components/..) Header.jsx, Footer.jsx.



## Toast.jsx, Navbar.jsx.



## 3. Pages(frontend/src/pages...)

### Home.jsx, Login.jsx



## Register.jsx, Procurement.jsx

```
1 // Register.jsx
2 import React, { useState } from 'react';
3 import { useNavigate } from 'react-router-dom';
4 import { Link, Button, Form, Input, Text, Password, Email, Phone, Select, Textarea } from 'react-bootstrap';
5 import { validateEmail, validatePhone, validatePassword } from 'utils';
6
7 const Register = () => {
8   const [user, setUser] = useState({});
9   const [login, setLogin] = useState('');
10   const [password, setPassword] = useState('');
11   const [confirmPassword, setConfirmPassword] = useState('');
12   const [errors, setErrors] = useState({});
13   const [loading, setLoading] = useState(false);
14
15   const handleSubmit = (e) => {
16     e.preventDefault();
17     if (!validateEmail(email) || !validatePhone(phone) || !validatePassword(password) || !validateConfirmPassword(confirmPassword)) {
18       setErrors({ email: 'Invalid email', phone: 'Invalid phone', password: 'Invalid password', confirmPassword: 'Invalid confirm password' });
19       return;
20     }
21     setLoading(true);
22     // Simulate API call
23     setTimeout(() => {
24       // Success
25       alert('Registration successful!');
26       navigate('/login');
27     }, 2000);
28     // Failure
29     // setErrors({ message: 'Registration failed' });
30   };
31
32   const handleLogin = (e) => {
33     e.preventDefault();
34     // Simulate API call
35     setTimeout(() => {
36       // Success
37       alert('Login successful!');
38       navigate('/dashboard');
39     }, 2000);
40     // Failure
41     // setErrors({ message: 'Login failed' });
42   };
43
44   return (
45     <div>
46       <h2>Register</h2>
47       <form>
48         <input type="text" value={email} onChange={e => setEmail(e.target.value)} />
49         <input type="text" value={phone} onChange={e => setPhone(e.target.value)} />
50         <input type="password" value={password} onChange={e => setPassword(e.target.value)} />
51         <input type="password" value={confirmPassword} onChange={e => setConfirmPassword(e.target.value)} />
52         <button type="button" value={login} onClick={handleLogin} />
53         <button type="button" value={register} onClick={handleSubmit} />
54       </form>
55     </div>
56   );
57 };
58
59 export default Register;
```

```
1 // Procurement.jsx
2 import React, { useState } from 'react';
3 import { useNavigate } from 'react-router-dom';
4 import { Link, Button, Form, Input, Text, Password, Email, Phone, Select, Textarea } from 'react-bootstrap';
5 import { validateEmail, validatePhone, validatePassword } from 'utils';
6
7 const Procurement = () => {
8   const [user, setUser] = useState({});
9   const [supplier, setSupplier] = useState('');
10   const [product, setProduct] = useState('');
11   const [quantity, setQuantity] = useState('');
12   const [price, setPrice] = useState('');
13   const [status, setStatus] = useState('');
14   const [errors, setErrors] = useState({});
15   const [loading, setLoading] = useState(false);
16
17   const handleSubmit = (e) => {
18     e.preventDefault();
19     if (!validateEmail(email) || !validatePhone(phone) || !validatePassword(password) || !validateSupplier(supplier) || !validateProduct(product) || !validateQuantity(quantity) || !validatePrice(price) || !validateStatus(status)) {
20       setErrors({ email: 'Invalid email', phone: 'Invalid phone', password: 'Invalid password', supplier: 'Invalid supplier', product: 'Invalid product', quantity: 'Invalid quantity', price: 'Invalid price', status: 'Invalid status' });
21       return;
22     }
23     setLoading(true);
24     // Simulate API call
25     setTimeout(() => {
26       // Success
27       alert('Procurement successful!');
28       navigate('/dashboard');
29     }, 2000);
30     // Failure
31     // setErrors({ message: 'Procurement failed' });
32   };
33
34   return (
35     <div>
36       <h2>Procurement</h2>
37       <form>
38         <input type="text" value={email} onChange={e => setEmail(e.target.value)} />
39         <input type="text" value={phone} onChange={e => setPhone(e.target.value)} />
40         <input type="password" value={password} onChange={e => setPassword(e.target.value)} />
41         <input type="text" value={supplier} onChange={e => setSupplier(e.target.value)} />
42         <input type="text" value={product} onChange={e => setProduct(e.target.value)} />
43         <input type="text" value={quantity} onChange={e => setQuantity(e.target.value)} />
44         <input type="text" value={price} onChange={e => setPrice(e.target.value)} />
45         <input type="text" value={status} onChange={e => setStatus(e.target.value)} />
46         <button type="button" value={submit} onClick={handleSubmit} />
47       </form>
48     </div>
49   );
50 };
51
52 export default Procurement;
```

## Sales.jsx, Warehouse.jsx

```
1 // Sales.jsx
2 import React, { useState } from 'react';
3 import { useNavigate } from 'react-router-dom';
4 import { Link, Button, Form, Input, Text, Password, Email, Phone, Select, Textarea } from 'react-bootstrap';
5 import { validateEmail, validatePhone, validatePassword } from 'utils';
6
7 const Sales = () => {
8   const [user, setUser] = useState({});
9   const [customer, setCustomer] = useState('');
10   const [product, setProduct] = useState('');
11   const [quantity, setQuantity] = useState('');
12   const [price, setPrice] = useState('');
13   const [status, setStatus] = useState('');
14   const [errors, setErrors] = useState({});
15   const [loading, setLoading] = useState(false);
16
17   const handleSubmit = (e) => {
18     e.preventDefault();
19     if (!validateEmail(email) || !validatePhone(phone) || !validatePassword(password) || !validateCustomer(customer) || !validateProduct(product) || !validateQuantity(quantity) || !validatePrice(price) || !validateStatus(status)) {
20       setErrors({ email: 'Invalid email', phone: 'Invalid phone', password: 'Invalid password', customer: 'Invalid customer', product: 'Invalid product', quantity: 'Invalid quantity', price: 'Invalid price', status: 'Invalid status' });
21       return;
22     }
23     setLoading(true);
24     // Simulate API call
25     setTimeout(() => {
26       // Success
27       alert('Sales successful!');
28       navigate('/dashboard');
29     }, 2000);
30     // Failure
31     // setErrors({ message: 'Sales failed' });
32   };
33
34   return (
35     <div>
36       <h2>Sales</h2>
37       <form>
38         <input type="text" value={email} onChange={e => setEmail(e.target.value)} />
39         <input type="text" value={phone} onChange={e => setPhone(e.target.value)} />
40         <input type="password" value={password} onChange={e => setPassword(e.target.value)} />
41         <input type="text" value={customer} onChange={e => setCustomer(e.target.value)} />
42         <input type="text" value={product} onChange={e => setProduct(e.target.value)} />
43         <input type="text" value={quantity} onChange={e => setQuantity(e.target.value)} />
44         <input type="text" value={price} onChange={e => setPrice(e.target.value)} />
45         <input type="text" value={status} onChange={e => setStatus(e.target.value)} />
46         <button type="button" value={submit} onClick={handleSubmit} />
47       </form>
48     </div>
49   );
50 };
51
52 export default Sales;
```

```
1 // Warehouse.jsx
2 import React, { useState } from 'react';
3 import { useNavigate } from 'react-router-dom';
4 import { Link, Button, Form, Input, Text, Password, Email, Phone, Select, Textarea } from 'react-bootstrap';
5 import { validateEmail, validatePhone, validatePassword } from 'utils';
6
7 const Warehouse = () => {
8   const [user, setUser] = useState({});
9   const [warehouse, setWarehouse] = useState('');
10   const [product, setProduct] = useState('');
11   const [quantity, setQuantity] = useState('');
12   const [price, setPrice] = useState('');
13   const [status, setStatus] = useState('');
14   const [errors, setErrors] = useState({});
15   const [loading, setLoading] = useState(false);
16
17   const handleSubmit = (e) => {
18     e.preventDefault();
19     if (!validateEmail(email) || !validatePhone(phone) || !validatePassword(password) || !validateWarehouse(warehouse) || !validateProduct(product) || !validateQuantity(quantity) || !validatePrice(price) || !validateStatus(status)) {
20       setErrors({ email: 'Invalid email', phone: 'Invalid phone', password: 'Invalid password', warehouse: 'Invalid warehouse', product: 'Invalid product', quantity: 'Invalid quantity', price: 'Invalid price', status: 'Invalid status' });
21       return;
22     }
23     setLoading(true);
24     // Simulate API call
25     setTimeout(() => {
26       // Success
27       alert('Warehouse successful!');
28       navigate('/dashboard');
29     }, 2000);
30     // Failure
31     // setErrors({ message: 'Warehouse failed' });
32   };
33
34   return (
35     <div>
36       <h2>Warehouse</h2>
37       <form>
38         <input type="text" value={email} onChange={e => setEmail(e.target.value)} />
39         <input type="text" value={phone} onChange={e => setPhone(e.target.value)} />
40         <input type="password" value={password} onChange={e => setPassword(e.target.value)} />
41         <input type="text" value={warehouse} onChange={e => setWarehouse(e.target.value)} />
42         <input type="text" value={product} onChange={e => setProduct(e.target.value)} />
43         <input type="text" value={quantity} onChange={e => setQuantity(e.target.value)} />
44         <input type="text" value={price} onChange={e => setPrice(e.target.value)} />
45         <input type="text" value={status} onChange={e => setStatus(e.target.value)} />
46         <button type="button" value={submit} onClick={handleSubmit} />
47       </form>
48     </div>
49   );
50 };
51
52 export default Warehouse;
```

[illegible]

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import { ReturnsProvider } from '../context/ReturnsContext';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <ReturnsProvider>
          <App />
        </ReturnsProvider>
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

```
import { Routes, Route, Navigate } from 'react-router-dom';
import React from 'react';
import Router from '../components/Router';
import Footer from '../components/Footer';
import Checkout from '../components/Checkout';
import Toast from '../components/Toast';
import ProtectedRoute from '../components/ProtectedRoute';

import Home from './pages/HomePage';
import CustomerReturnsPage from './pages/CustomerReturnsPage';
import ReturnTrackingPage from './pages/ReturnTrackingPage';
import SupportPortalPage from './pages/SupportPortalPage';
import InspectionPage from './pages/InspectionPage';
import RefundPage from './pages/RefundPage';
import AdminAnalyticsPage from './pages/AdminAnalyticsPage';
import LoginPage from './pages/LoginPage';
import NotFoundPage from './pages/NotFoundPage';

import { useAuth } from '../context/AuthContext';
import './App.css';

export default function App() {
  const { isAuthenticated } = useAuth();

  // If unauthenticated, prompt ONLY the Sign In / Sign Up screen
  if (!isAuthenticated) {
    return (
      <div className="auth-fullscreen-wrapper">
        <LoginPage />
        <Toast />
      </div>
    );
  }

  return (
    <div className="app-wrapper">
      <NavigationHeader />
      <Router />
      <Footer />
    </div>
  );
}
```

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Returns Project / Titles</title>
  </head>
  <body>
    <div id="root">
      <script type="module" src="/src/main.jsx" />
    </div>
  </body>
</html>
```

```
<!-- @tailwindcss -->
import "tailwindcss";

/*
 * If you are seeing a broken layout, please restart your Vite dev server!
 * Stop it with Ctrl+C and run "npm run dev" again.
 */

@theme {
  --color-primary: #000000;
}

body {
  font-family: "Inter", -apple-system, sans-serif;
  background-color: #FFFFFF;
  color: #000000;
  --width: 100%;
}

h1, h2, h3, h4, h5, h6 {
  font-family: "Inter", -apple-system, sans-serif;
}

/* Custom Animations */
@keyframes fadeUp {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes shake {
  0%, 100% { transform: translate(0); }
  20% { transform: translate(-4px); }
  40% { transform: translate(4px); }
  60% { transform: translate(-4px); }
  80% { transform: translate(4px); }
}

.animate-fade-up {
  animation: fadeUp 0.5s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}

.animate-fade-in {
  animation: fadeIn 0.5s ease-out forwards;
}

.animate-shake {
  animation: shake 0.5s ease-in-out;
}

.delay-100 { animation-delay: 100ms; }
.delay-200 { animation-delay: 200ms; }
.delay-300 { animation-delay: 300ms; }
```

## 5. React Integration Files A.Axios Communication

### Client frontend/src/services/api.js

```
import axios from 'axios';

const API_URL = 'http://localhost:8080/api';

const axiosInstance = axios.create({
  baseURL: API_URL,
});

export default axiosInstance;
```

```
import axios from 'axios';

const API_URL = 'http://localhost:8080/api';

const axiosInstance = axios.create({
  baseURL: API_URL,
});

export default axiosInstance;
```

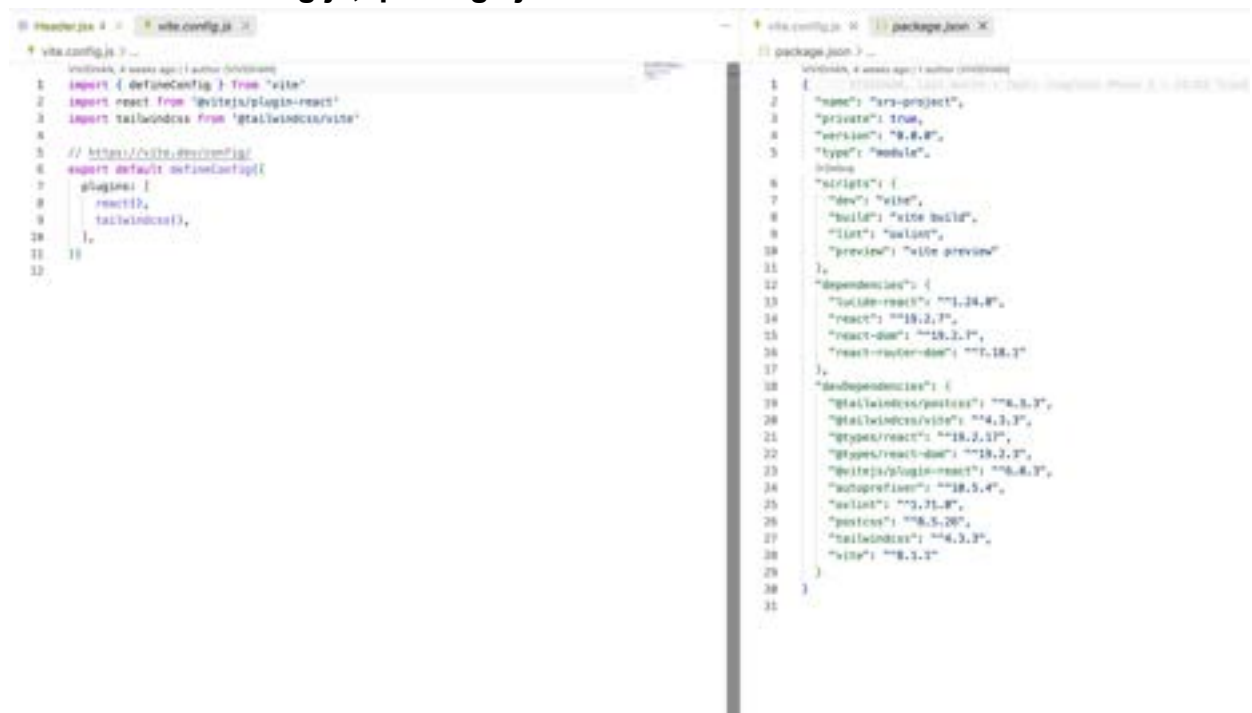
## frontend/src/context/AuthContext.jsx,



```
1 import React, { createContext, useState, useEffect, useContext } from 'react';
2 import { useNavigate } from 'react-router-dom';
3
4 const AuthContext = createContext(null);
5
6 export const AuthProvider = ({ children }) => {
7   const [user, setUser] = useState(null);
8   const [loading, setLoading] = useState(true);
9
10  useEffect(() => {
11    const storedUser = localStorage.getItem('fpo_user');
12    const token = localStorage.getItem('fpo_token');
13    if (storedUser && token) {
14      try {
15        setUser(JSON.parse(storedUser));
16      } catch (e) {
17        localStorage.removeItem('fpo_user');
18        localStorage.removeItem('fpo_token');
19      }
20    }
21    setLoading(false);
22  }, []);
23
24  const login = async (credentials) => {
25    setLoading(true);
26    const { identifier, password } = credentials;
27
28    try {
29      const response = await fetch('/api/auth/login', {
30        method: 'POST',
31        headers: {
32          'Content-Type': 'application/json',
33        },
34        body: JSON.stringify({ phone: identifier, password }),
35      });
36
37      const data = await response.json();
38
39      if (response.ok) {
40        const userData = { name: data.name, role: data.role, phone: identifier };
41        setUser(userData);
42        localStorage.setItem('fpo_user', JSON.stringify(userData));
43        localStorage.setItem('fpo_token', data.token);
44        setLoading(false);
45        return { success: true, user: userData };
46      } else {
47        setLoading(false);
48        return { success: false, message: data.error || 'Login failed' };
49      }
50    } catch (err) {
51      setLoading(false);
52      return { success: false, message: 'Network error connecting to backend' };
53    }
54  };
55
56  const logout = () => {
57    setUser(null);
58    localStorage.removeItem('fpo_user');
59    localStorage.removeItem('fpo_token');
```

## 6. Environment & Build Configuration

### Frontend/vite.config.js, package.json



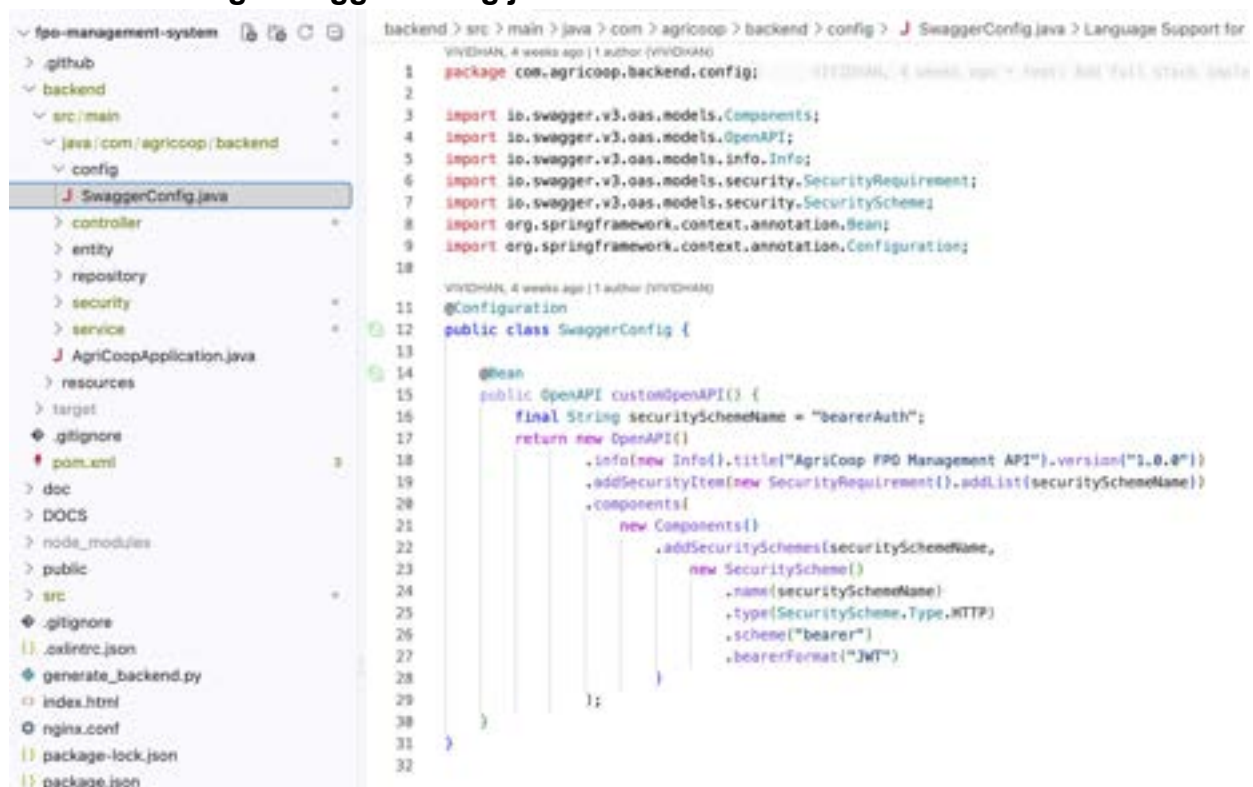
The screenshot shows two files in a VS Code editor. The left file is `vite.config.js` and the right file is `package.json`.

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3 import tailwindcss from 'tailwindcss/vite'
4
5 // https://vite.dev/config/
6 export default defineConfig({
7   plugins: [
8     react(),
9     tailwindcss(),
10   ],
11 })
```

```
1 {
2   "name": "ars-project",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "lint": "eslint",
10    "preview": "vite preview"
11  },
12  "dependencies": {
13    "react-dom": "18.2.0",
14    "react": "18.2.0",
15    "react-dom": "18.2.0",
16    "react-router-dom": "6.11.2"
17  },
18  "devDependencies": {
19    "@types/react": "18.2.3",
20    "@types/react-dom": "18.2.3",
21    "@types/react-dom": "18.2.3",
22    "@types/react-dom": "18.2.3",
23    "@types/react-dom": "18.2.3",
24    "eslint": "8.54.0",
25    "postcss": "8.4.38",
26    "tailwindcss": "3.4.3",
27    "vite": "5.1.1"
28  }
29 }
```

## 7. SpringBoot Backend Integration Files

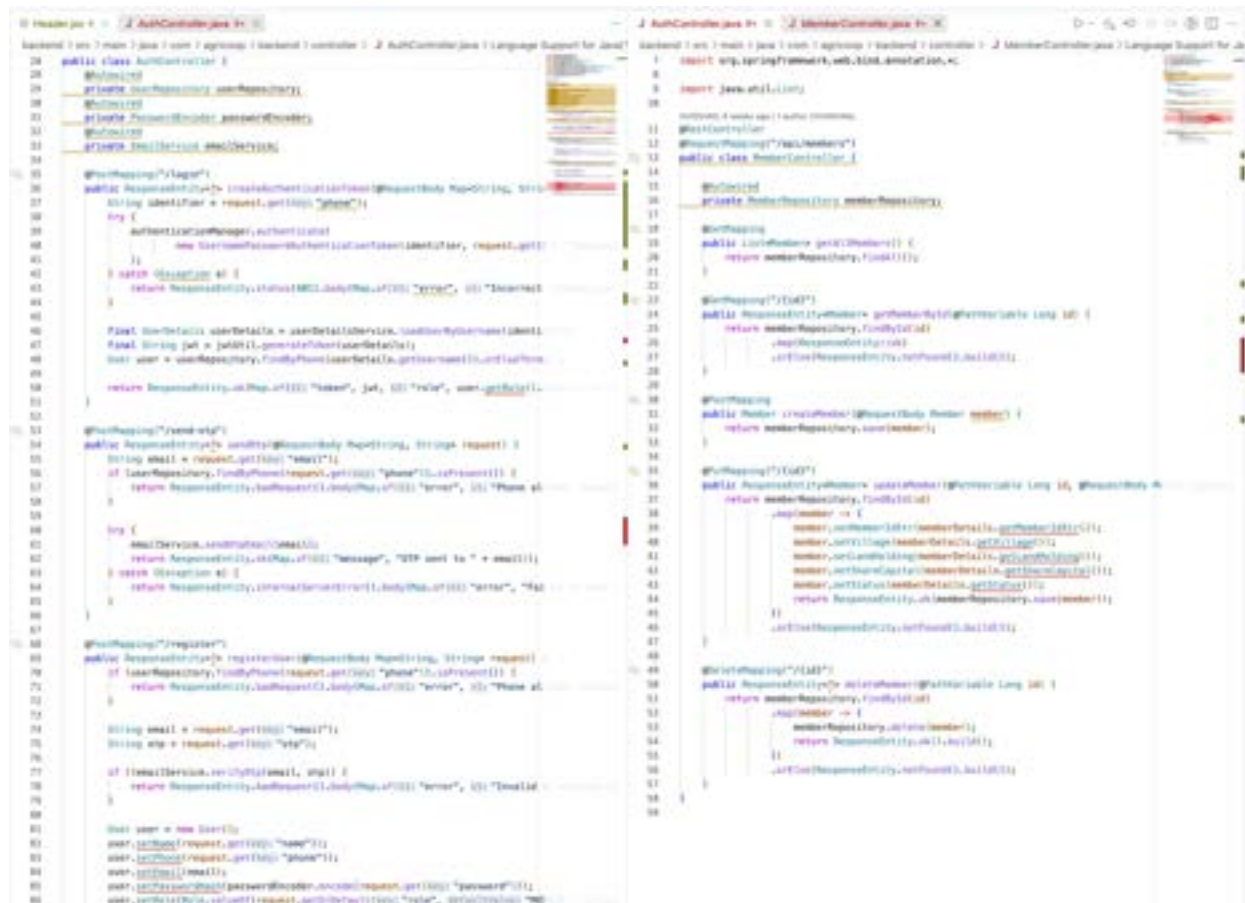
### Backend/config/ SwaggerConfig.java



The screenshot shows a VS Code editor with a file explorer on the left and the `SwaggerConfig.java` file open in the editor. The file explorer shows the project structure: `fpo-management-system` > `backend` > `src/main/java/com/agricoop/backend` > `config`. The `SwaggerConfig.java` file is highlighted in the explorer.

```
1 package com.agricoop.backend.config;
2
3 import io.swagger.v3.oas.models.Components;
4 import io.swagger.v3.oas.models.OpenAPI;
5 import io.swagger.v3.oas.models.info.Info;
6 import io.swagger.v3.oas.models.security.SecurityRequirement;
7 import io.swagger.v3.oas.models.security.SecurityScheme;
8 import org.springframework.context.annotation.Bean;
9 import org.springframework.context.annotation.Configuration;
10
11 @Configuration
12 public class SwaggerConfig {
13
14   @Bean
15   public OpenAPI customOpenAPI() {
16     final String securitySchemeName = "bearerAuth";
17     return new OpenAPI()
18       .info(new Info().title("AgrisCoop FPO Management API").version("1.0.0"))
19       .addSecurityItem(new SecurityRequirement().addList(securitySchemeName))
20       .components(
21         new Components()
22           .addSecuritySchemes(securitySchemeName,
23             new SecurityScheme()
24               .name(securitySchemeName)
25               .type(SecurityScheme.Type.HTTP)
26               .scheme("bearer")
27               .bearerFormat("JWT")
28             )
29       );
30   }
31 }
32
```

### Backend/controller/AuthController.java , ReturnRequestController.java



## 8. Database Configuration Files:

## Application.properties



9. OUTPUT

Login Page:



## Welcome Back

Sign in to your AgriCoop portal

☐ QUICK TEST LOGIN BY ROLE

Admin

FPO Manager

Collection Agent

Board Member

Farmer Member

Email, Phone, or Member ID

 admin@fpo.com 

Password

 ..... 

[Forgot Password?](#)

Sign In →

Don't have an account? [Register New Account](#)

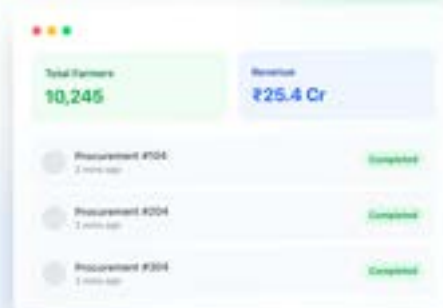
AgriCoop 2.0 is now live

# Digitally Transform Your Farmer Producer Organization

Manage members, finance, warehouse, procurement, sales, and analytics — all from one unified, enterprise-grade platform.

Request Demo

Explore Features



TRUSTED BY LEADING AGRICULTURAL COOPERATIVES

AgriCorp

KisanMandi

MahafPO

GreenProduce

BharatFarm

## Everything you need to scale

A modular architecture designed specifically for the unique challenges of modern FPOs.



### Member Management

Digitally onboard farmers, track land holding, manage share capital, and keep KYC records centralized.



### Intelligent Warehousing

Track storage facilities, available capacity and current stock in real-time with automated alerts.



### Market Linkage

Aggregate produce and sell seamlessly to APMC, e-NAM, and private buyers with payment tracking.



### Audit & Compliance

Generate instantaneous compliance reports, Role-Based Access Control and JWT security built-in.



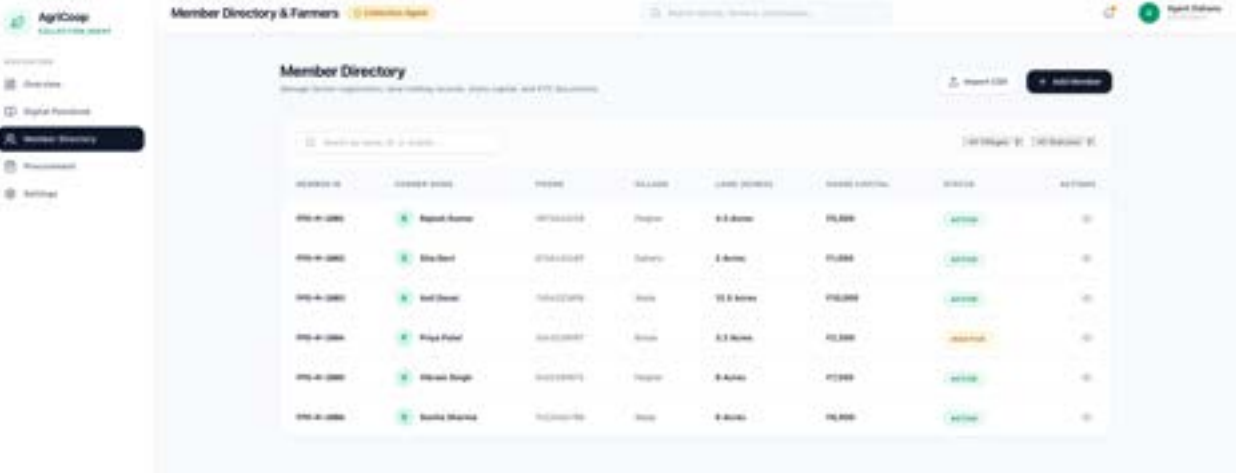
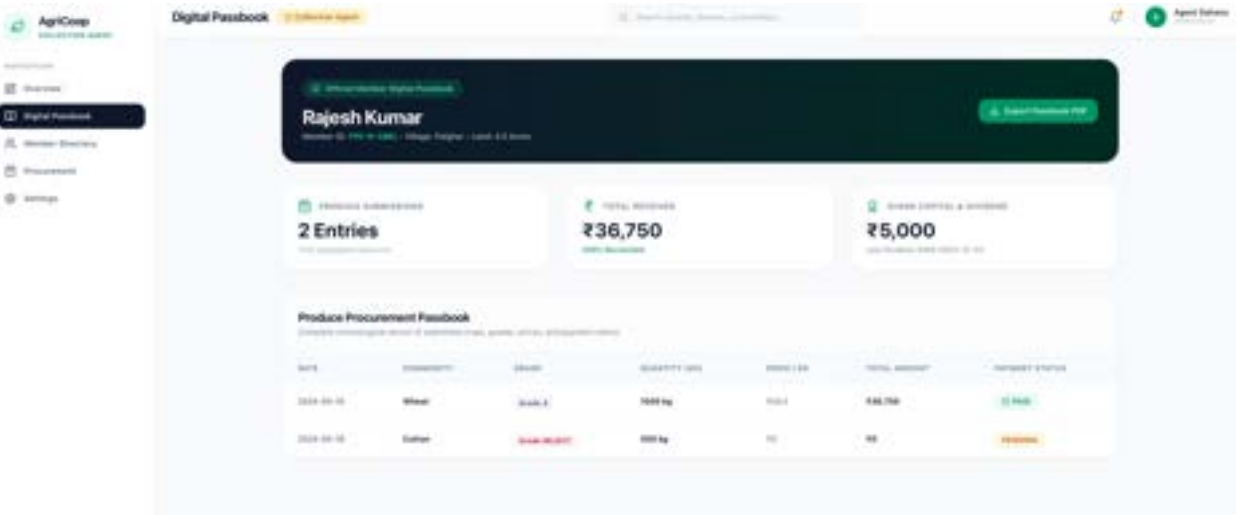
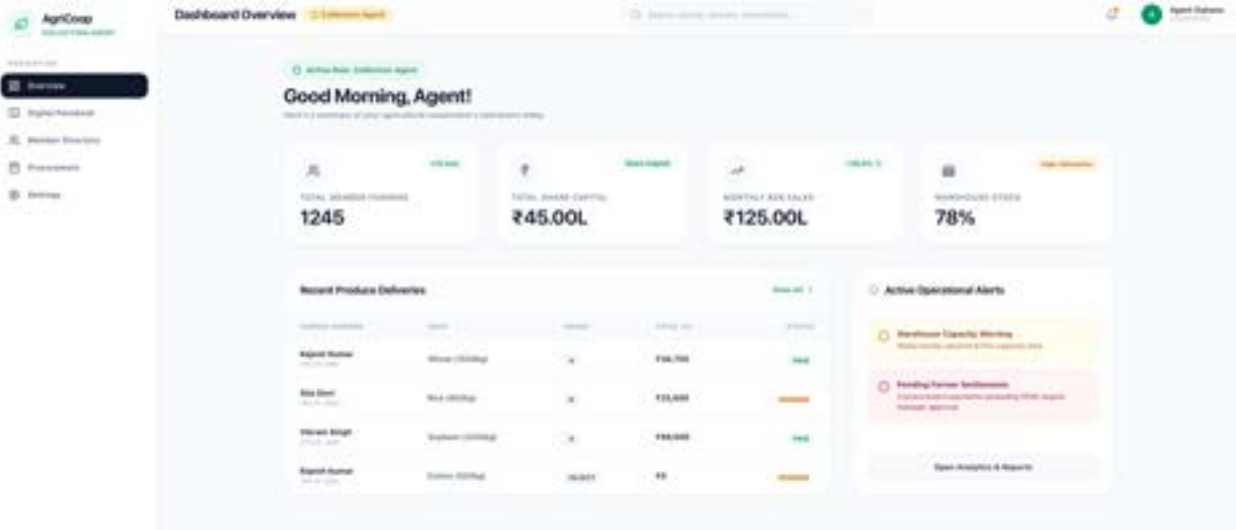
### Mobile Ready

Fully responsive progressive web app allows collection agents to operate directly from the field.



### Financial Tracking

Automated dividend calculation and real-time transaction reconciliation for absolute transparency.



The screenshot displays the 'Produce Aggregation & Procurement' dashboard. At the top, there's a header with the AgriCoop logo and navigation tabs: Overview, Digital Payments, and Procurement (which is active). Below the header, the main title 'Produce Aggregation & Procurement' is followed by a subtitle: 'Monitor aggregation orders, manage procurement, quality grading, and logistics.' A 'Recent Procurement' button is located on the right. The main content area features a table with the following data:

| Date       | Product Name    | Quantity | Grade   | Price  | Total Amount | Status  |
|------------|-----------------|----------|---------|--------|--------------|---------|
| 2023-04-01 | AgriCoop Tomato | 1000 kg  | Grade A | 150.00 | 150,000      | On Hold |
| 2023-04-02 | White Onion     | 500 kg   | Grade B | 120.00 | 60,000       | On Hold |
| 2023-04-03 | Yellow Onion    | 2000 kg  | Grade A | 100.00 | 200,000      | On Hold |
| 2023-04-04 | AgriCoop Potato | 800 kg   | Grade A | 75.00  | 60,000       | On Hold |
| 2023-04-05 | Red Onion       | 1000 kg  | Grade A | 120.00 | 120,000      | On Hold |

The screenshot displays the 'System Settings' page in the AgriCloud application. The page is organized into three main sections:

- Profile Details:** This section includes fields for 'Name', 'Email', and 'Password', along with a 'Save' button.
- Notification Channels:** This section lists 'Email Notifications' and 'SMS Notifications', each with a toggle switch to enable or disable notifications.
- Account Security:** This section includes a 'Change Password' button.

**AgriCoop Farm Manager**

**Dashboard Overview**

**Good Morning, Manager!**

Here's a snapshot of your agricultural operations' essential data.

| Icon           | Label                  | Value    |
|----------------|------------------------|----------|
| Person icon    | TOTAL WORKERS EMPLOYED | 1245     |
| ₹ icon         | TOTAL HARVEST CAPACITY | ₹45.00L  |
| Truck icon     | MONTHLY PRODUCE SALES  | ₹125.00L |
| Bar chart icon | WORKLOAD STATUS        | 78%      |

**Recent Produce Deliveries**

| Delivery Record | Item           | Weight  | Price (₹) | Status    |
|-----------------|----------------|---------|-----------|-----------|
| Export Batch    | Organic Apples | 1500 kg | ₹120,000  | Shipped   |
| Wholesale       | Box Apples     | 800 kg  | ₹105,000  | Delivered |
| Wholesale       | Organic Apples | 1200 kg | ₹95,000   | Shipped   |
| Export Batch    | Box Apples     | 500 kg  | ₹0        | Delivered |

**Active Operational Alerts**

- Workload Capacity Warning:** Workload exceeds 80% capacity.
- Pending Farmer Settlements:** 3 small farmers awaiting pending 10% export manager approval.

[Open Analytics & Reports](#)

## Warehouse &amp; Stock Inventory

Real-time monitoring of warehouse facilities, inventory status, and stock levels.

Add Warehouse Facility

Palghar Central Hub

Inventory: 100 MT

Current Stock: 100 MT

Stock Capacity: 100 MT

Utilization: 100%

Report Stock List

Wadi Facility

Inventory: 100 MT

Current Stock: 100 MT

Stock Capacity: 100 MT

Utilization: 100%

Report Stock List

Daham Cold Storage

Inventory: 100 MT

Current Stock: 100 MT

Stock Capacity: 100 MT

Utilization: 100%

Report Stock List

Bolpur Logistics Point

Inventory: 100 MT

Current Stock: 100 MT

Stock Capacity: 100 MT

Utilization: 100%

Report Stock List

## Analytics &amp; Reporting Dashboard

Real-time analytics of inventory, sales, and compliance metrics.

This Month: 100 MT

Total Inventory: ₹11.46L

Inventory Status: 8.0 MT

Inventory Value: ₹909.6k

Inventory Status: 78%

Commodity Procurement vs Sales

Wheat (Grade A): 100 MT Procured / 100 MT Sold

Rice (Grade A): 100 MT Procured / 100 MT Sold

Rice (Grade B): 100 MT Procured / 100 MT Sold

Predictive Demand & Risk Indicators

High Demand Forecast: Analysis

Inventory Holding Risk: Alert

## Compliance &amp; Audit Repository

Manage all compliance documents, audit logs, and generate reports.

Document Repository

Upload Document

## Official IPO Documents

Manage all IPO documents, including IPO forms, prospectuses, and other related documents.

| Document ID | Category     | Status      | Created Date | Updated Date | Actions              |
|-------------|--------------|-------------|--------------|--------------|----------------------|
| 1001        | Registration | Completed   | 2023-10-01   | 2023-10-01   | <a href="#">View</a> |
| 1002        | Listing      | In Progress | 2023-10-02   | 2023-10-02   | <a href="#">View</a> |
| 1003        | Compliance   | Completed   | 2023-10-03   | 2023-10-03   | <a href="#">View</a> |
| 1004        | Audit        | Completed   | 2023-10-04   | 2023-10-04   | <a href="#">View</a> |

## 10. DATABASE:

```
[> mysql -u root -p'Root@12345'
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 19
Server version: 9.7.1 Homebrew

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE agricoop;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
```

32 rows in set (0.021 sec)

```
mysql> use agricoop ;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

```
Database changed
mysql> show tables;
```

|                    |
|--------------------|
| Tables_in_agricoop |
| members            |
| procurements       |
| users              |

3 rows in set (0.012 sec)

```
mysql> SELECT id, name, email, role FROM users;
```

| id | name     | email             | role  |
|----|----------|-------------------|-------|
| 1  | Vividhan | vividhan@test.com | ADMIN |

1 row in set (0.001 sec)

```
mysql> SELECT id, name, email, role FROM users;
```

| id | name     | email             | role  |
|----|----------|-------------------|-------|
| 1  | Vividhan | vividhan@test.com | ADMIN |

1 row in set (0.001 sec)

```
mysql> █
```